

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра автоматизированных систем управления (АСУ)

Отчёт по Научно-исследовательской работе

Студент гр. 435-М1
_____ Д.П. Андреев
«__»
_____ 2026
Проверил
Профессор каф. АСУ,
д.т.н
_____ М.Ю. Катаев
«__» _____ 2026

Томск 2026

Оглавление

Введение	3
1 ГЛАВА.....	9
1.1 Обзор предметной области.....	9
1.2 БПЛА, как летательный аппарат	10
1.3 Инерциальное устройство на борту БПЛА.....	12
1.4 Логи́рование полета	13
1.5 Обработка данных инерциального устройства	14
1.6 Описание программ аналогов	16
2 ГЛАВА. РАЗРАБОТКА АРХИТЕКТУРЫ И МЕТОДОВ ПОСТРОЕНИЯ ЦИФРОВОГО ДВОЙНИКА.....	19
2.1 Получение данных БПЛА.....	19
2.1.1 Классификация и структура данных БПЛА	19
2.1.2 Источники данных: реальный полёт (физический БПЛА)	21
2.1.3 Источники данных: виртуальный полёт (цифровой двойник)	22
2.2 Математическая модель динамики БПЛА.....	23
2.2.1 Системы координат и переменные состояния.....	23
2.2.2 Уравнения динамики поступательного движения.....	24
2.2.3 Уравнения динамики вращательного движения	25
2.2.4 Кинематические соотношения Эйлера.....	25
2.2.5 Модель силовой установки и управление	25
2.2.6 Моделирование аэродинамических помех	26
2.3 Архитектура программного комплекса.....	26
Список использованных источников	35

Введение

Разработка и внедрение беспилотных авиационных систем (БПЛА) стали одним из ключевых технологических трендов последнего десятилетия. Области их применения стремительно расширяются: от мониторинга и картографии, доставки грузов, сельского хозяйства и инспекции инфраструктуры до задач обороны и безопасности. Однако массовая эксплуатация БПЛА сдерживается комплексом вызовов, связанных с обеспечением безопасности, надежности и предсказуемости их работы в сложных и динамичных условиях [1].

Традиционный цикл разработки, основанный на создании физических прототипов и их натурных испытаниях, становится «узким местом». Он характеризуется:

- Высокой стоимостью (риск уничтожения дорогостоящего аппарата, затраты на полигоны, логистику).
- Длительностью (зависимость от погодных условий, бюрократические процедуры получения разрешений на полеты).
- Ограниченной полнотой тестирования (невозможно безопасно воспроизвести все критические и аварийные ситуации в реальности).
- Сложностью отладки и анализа (процессы, происходящие в полете, фиксируются ограниченным набором датчиков, что затрудняет постфактум анализ сбоев).

В этом контексте концепция цифрового двойника (Digital Twin) приобретает исключительную актуальность. Цифровой двойник БПЛА – это не просто симулятор, а высокоточная виртуальная копия, непрерывно обменивающаяся данными с физическим объектом и способная предсказывать его состояние и поведение [2]. Актуальность создания и использования таких систем для тестирования обусловлена:

1. Требованиями к ускорению R&D-циклов: Необходимость быстрого вывода на рынок конкурентоспособных продуктов требует переноса основной части тестирования и отладки в виртуальную среду [3].

2. Развитием автономных систем: Тестирование интеллектуальных алгоритмов на основе машинного обучения (например, для облета препятствий или посадки в сложных условиях) требует миллионов часов виртуальных полетов для накопления опыта [4].

3. Необходимостью валидации в условиях, близких к предельным: Моделирование отказов оборудования, экстремальных погодных явлений (обледенение, сильная турбулентность), электромагнитных помех – необходимо для сертификации и подтверждения надежности.

4. Цифровизацией жизненного цикла изделия: Цифровой двойник становится единым источником истины для всех этапов – от проектирования и испытаний до эксплуатации, технического обслуживания и модернизации [5].

Таким образом, исследования и разработки в области создания адекватных, физически достоверных и функционально полных цифровых двойников для БПЛА являются стратегически важными для технологического суверенитета и лидерства в высокотехнологичных отраслях.

Четкое разграничение объекта и предмета исследования является основой для корректной постановки научной или инженерной задачи.

Объект исследования – это процесс разработки, тестирования и валидации комплексных систем управления и навигации беспилотных летательных аппаратов мультироторного (вертолетного) или самолетного типа. В более широком смысле, объектом является жизненный цикл БПЛА на этапе виртуальных испытаний, где цифровая модель служит основной средой для проверки работоспособности и эффективности всех подсистем.

Предмет исследования – это архитектура, методы и инструменты построения цифрового двойника БПЛА, ориентированного на задачи

всестороннего тестирования его математической и программной моделей. Конкретными аспектами предмета являются:

— Методы математического моделирования аэродинамики и динамики полета БПЛА, обеспечивающие необходимый баланс между точностью и вычислительной эффективностью [6].

— Подходы к эмуляции работы бортовых инерциальных и других сенсоров, включая моделирование характерных погрешностей и помех.

— Алгоритмы и инструменты для синхронной записи (логирования) и последующего анализа данных как виртуального, так и реального полета с целью взаимной валидации.

— Способы интеграции моделей (динамики, среды, датчиков) и реального аппаратного обеспечения (полетных контроллеров) в единый контур тестирования (HIL, SIL).

— Критерии и метрики для оценки адекватности цифрового двойника и эффективности тестируемых в нем алгоритмов.

Таким образом, исследование фокусируется не на конструкции конкретного БПЛА, а на методологии создания и использования его виртуального прототипа как основного инструмента инженерного анализа.

Цель исследования – разработать методологию и провести апробацию программного комплекса цифрового двойника для всестороннего тестирования моделей динамики, систем управления и навигации беспилотного летательного аппарата, обеспечивающего значительное снижение затрат и рисков на этапе прототипирования и испытаний.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Аналитико-поисковая задача: провести анализ предметной области, существующих подходов к моделированию динамики БПЛА, эмуляции сенсоров и инструментальных средств (ПО) для построения цифровых двойников. Выявить их преимущества, недостатки и области применимости.

2. Методологическая задача: сформировать архитектуру цифрового двойника, определяющую состав взаимосвязанных моделей (динамики, среды, датчиков), интерфейсы между ними, а также контуры тестирования (Model-in-the-Loop, Software-in-the-Loop, Hardware-in-the-Loop) [7][8][9].

3. Задача моделирования: разработать или адаптировать нелинейную математическую модель динамики и аэродинамики для выбранного типа БПЛА (например, мультикоптера или самолета). Разработать детализированные модели ключевых бортовых датчиков (IMU, GPS) с учетом их характерных погрешностей [10][11].

4. Инструментальная задача: реализовать или интегрировать на базе выбранного программного фреймворка (например, ROS/Gazebo или MATLAB/Simulink) работоспособный прототип цифрового двойника с функциями логирования данных [12][13].

5. Экспериментально-аналитическая задача: провести серию вычислительных экспериментов по тестированию в созданном цифровом двойнике типовых алгоритмов управления и навигации. Оценить адекватность модели путем сравнения результатов симуляции с эталонными данными или, при наличии, с логами реальных полетов. Продемонстрировать практическую значимость подхода на конкретных сценариях (например, тест на устойчивость, отработка полета по маршруту, моделирование отказа датчика).

Решение этих задач в комплексе позволит не только создать конкретный инструмент, но и отработать универсальную последовательность действий для построения цифровых двойников БПЛА различного назначения.

Результаты исследования, направленные на создание и применение цифрового двойника для тестирования моделей БПЛА, обладают высокой практической значимостью для широкого круга стейкхолдеров:

1. Для разработчиков БПЛА и систем управления:

— Сокращение сроков и стоимости разработки: Перенос основной массы тестов в виртуальную среду ускоряет итерации по улучшению

конструкции и алгоритмов, минимизирует необходимость в дорогостоящих физических прототипах на ранних стадиях.

— Повышение качества и надежности продукции: Возможность проведения исчерпывающих тестов в экстремальных и аварийных ситуациях позволяет выявлять и устранять скрытые дефекты на этапе проектирования.

— Упрощение отладки и анализа: Детальное логирование всех внутренних переменных модели и доступ к любому «виртуальному датчику» обеспечивают беспрецедентную глубину анализа причин нештатного поведения.

2. Для научно-исследовательских коллективов и образовательных учреждений:

— Создание воспроизводимой экспериментальной базы: Цифровой двойник предоставляет безопасную и контролируемую среду для исследований в области робототехники, теории управления, искусственного интеллекта и компьютерного зрения.

— Эффективный инструмент для обучения: позволяет студентам и инженерам изучать сложную динамику БПЛА, принципы работы автопилотов и отрабатывать навыки пилотирования без риска поломки аппаратуры.

— Генерация синтетических данных: может использоваться для создания больших размеченных датасетов, необходимых для обучения нейросетевых алгоритмов (например, для навигации по визуальным ориентирам), когда сбор реальных данных затруднен.

3. Для эксплуатантов и регуляторов:

— Предполетная проверка миссий: позволяет операторам «прогнать» планируемый полетный план в симуляции для выявления потенциальных рисков (например, пересечения зон ограниченной видимости, недостатка энергии).

— Расследование инцидентов: может служить инструментом для реконструкции событий реального полета на основе черного ящика (логов) и анализа «что, если» в идентичных условиях.

— Поддержка процессов сертификации: может предоставлять доказательную базу для проверки соответствия БПЛА нормативным требованиям по безопасности и надежности, особенно для критических сценариев, которые сложно продемонстрировать в натуре.

Таким образом, внедрение методологии цифровых двойников способно кардинально трансформировать индустрию БПЛА, сделав процессы разработки и эксплуатации более быстрыми, дешевыми, безопасными и основанными на данных.

1 ГЛАВА

1.1 Обзор предметной области

Концепция цифрового двойника (Digital Twin) представляет собой динамическую виртуальную копию физического объекта или системы, которая имитирует его структуру, состояние и поведение в реальном времени на основе данных с датчиков. В области беспилотных авиационных систем (БПЛА) цифровой двойник становится ключевым инструментом для проектирования, тестирования, валидации и сопровождения на всех этапах жизненного цикла.

Основная цель создания цифрового двойника для тестирования модели БПЛА – минимизация рисков и затрат, связанных с натурными испытаниями. Натурные полеты сопряжены с высокими финансовыми издержками, риском потери аппарата, зависимостью от погодных условий и нормативных ограничений. Цифровой двойник позволяет проводить интенсивные и повторяемые тесты в безопасной виртуальной среде. Это включает в себя:

- Отладку и верификацию алгоритмов управления и навигации.
- Моделирование полета в различных, в том числе экстремальных, условиях (турбулентность, отказ датчиков, потеря связи).
- Оценку летно-технических характеристик до создания физического прототипа.
- Обучение автономных систем с использованием методов машинного обучения и искусственного интеллекта.
- Предполетную проверку миссий и анализ их выполнения.

Цифровой двойник БПЛА, как правило, является комплексной системой, состоящей из нескольких взаимосвязанных моделей:

1. **Модель динамики и аэродинамики (Dynamics Model):** физически точная математическая модель, описывающая движение аппарата в пространстве под действием сил и моментов [14].

2. **Модель силовой установки и систем (Plant Model):** моделирует работу двигателей, сервоприводов, систем энергопитания [15].

3. **Модель датчиков (Sensor Model):** воспроизводит работу инерциальных измерительных блоков (IMU), GPS-приемников, барометров, добавляя характерные для реальных устройств шумы, смещения и задержки [16].

4. **Модель среды (Environment Model):** включает модели атмосферы (ветер, плотность воздуха), гравитационного поля, рельефа местности, электромагнитной обстановки (GPS-помехи) [17].

5. **Модель системы управления (Control System Model):** алгоритмы автопилота, стабилизации, следования по маршруту [18].

Интеграция этих моделей в единый контур моделирования с возможностью подключения реального аппаратного обеспечения (технология Hardware-in-the-Loop, HIL) формирует полноценный цифровой двойник. Таким образом, предметная область лежит на стыке аэромеханики, теории управления, моделирования, робототехники и data science.

1.2 БПЛА, как летательный аппарат

Беспилотный летательный аппарат (БПЛА) – это летательный аппарат, выполняющий полет без экипажа на борту, управляемый оператором дистанционно или автономно бортовыми компьютерами по заданной программе. Для целей моделирования и создания цифрового двойника БПЛА рассматривается как сложная динамическая система, описываемая законами механики и аэродинамики.

Ключевые аспекты, которые должны быть отражены в модели БПЛА:

— **Конструкция и масса:** геометрия планера (крыло, фюзеляж, оперение), распределение масс, момент инерции. Эти параметры напрямую влияют на устойчивость и управляемость.

— **Аэродинамические характеристики:** определяются геометрией и описываются набором безразмерных коэффициентов (подъемной силы,

лобового сопротивления, моментов тангажа, рыскания и крена). Эти коэффициенты являются функциями углов атаки и скольжения, числа Рейнольдса и Маха. Для их получения используются методы вычислительной гидродинамики (CFD), испытания в аэродинамической трубе или приближенные аналитические методы [19].

— **Динамика движения:** Движение БПЛА описывается системой дифференциальных уравнений в трехмерном пространстве. Обычно используются два связанных набора уравнений:

1. Уравнения движения центра масс (поступательное движение) в связанной системе координат.
2. Уравнения Эйлера (вращательное движение) для описания изменения угловой ориентации аппарата.

— **Силовая установка:** Модель двигателя (электрический бесколлекторный или ДВС) и воздушного винта. Важно моделировать зависимость создаваемой тяги и потребляемого момента от скорости вращения, набегающего потока воздуха и высоты полета.

— **Система управления:** формирует управляющие сигналы для рулевых машинок (элеронов, рулей высоты и направления) на основе заданных целей и текущей информации с датчиков. В цифровом двойнике эта система может быть представлена как виртуальным алгоритмом (Model-in-the-Loop), так и реальным полетным контроллером (Hardware-in-the-Loop).

Точность модели динамики БПЛА является фундаментом для всего цифрового двойника. Упрощенные модели (например, модель точки массы) подходят для планирования маршрутов, но для тестирования системы стабилизации и автономного пилотажа необходима полная нелинейная 6-степенная модель, учитывающая взаимовлияние продольного и бокового движений.

1.3 Инерциальное устройство на борту БПЛА

Инерциальный измерительный блок (Inertial Measurement Unit, IMU) – это ключевой датчик ориентации и навигации на борту БПЛА. Он предоставляет информацию об угловых скоростях и линейных ускорениях аппарата, что является основой для оценки его позы (ориентации и положения). Стандартный IMU включает в себя два типа датчиков:

— **Трехосевой гироскоп**: измеряет угловую скорость вращения вокруг трех осей (крен, тангаж, рыскание).

— **Трехосевой акселерометр**: измеряет кажущееся линейное ускорение по трем осям, включая гравитационное ускорение.

В современных БПЛА IMU часто входит в состав более сложного устройства – инерциальной навигационной системы (Inertial Navigation System, INS), которая интегрирует данные IMU для расчета скорости, положения и ориентации. Важным дополнением к IMU является магнитометр (трехосевой компас), измеряющий вектор магнитного поля Земли для определения курса, и барометр для оценки высоты.

Для целей цифрового двойника критически важно не просто использовать идеальные значения угловых скоростей и ускорений, получаемые из модели динамики, а эмулировать реалистичные выходные сигналы IMU. Это включает моделирование следующих эффектов и ошибок:

— **Смещение нуля (Bias)**: Постоянная или медленно меняющаяся добавка к истинному сигналу. Особенно критично для гироскопов, так как их смещение приводит к накоплению ошибки интегрированием при расчете углов.

— **Шум (Noise)**: Высокочастотные случайные колебания сигнала. Моделируется как белый шум или более сложными процессами (например, шумом Калмана).

— **Масштабный фактор и не ортогональность осей**: Нелинейность и перекрестное влияние осей датчиков.

— **Температурные дрейфы:** Изменение параметров датчиков в зависимости от температуры.

— **Вибрационные воздействия:** Воздействие высокочастотных вибраций от двигателя и пропеллера, которые могут наводить помехи на акселерометры.

Точная модель IMU позволяет протестировать и отладить алгоритмы фильтрации (в первую очередь, фильтр Калмана) в условиях, максимально приближенных к реальным. Это обеспечивает корректную работу алгоритмов оценки ориентации (Attitude and Heading Reference System, AHRS) и навигации в цифровом двойнике и гарантирует, что эти алгоритмы будут устойчивы к погрешностям реальных датчиков.

1.4 Логирование полета

Логирование (запись) данных полета – это процесс сохранения всех критически важных параметров БПЛА во время реального или виртуального полета в цифровом виде. В контексте цифрового двойника логирование выполняет несколько ключевых функций:

1. Верификация и валидация моделей: Записанные данные с реальных полетов (с реального БПЛА-прототипа) используются для калибровки и уточнения моделей динамики, аэродинамики и датчиков цифрового двойника. Сравнение поведения двойника и реального аппарата на идентичных управляющих воздействиях – основной метод валидации.

2. Воспроизведение и отладка: Логи позволяют воспроизвести конкретный полет или его опасный эпизод в симуляторе для детального анализа причин нештатной ситуации, отладки алгоритмов управления и проверки исправлений.

3. Обучение и тестирование алгоритмов ИИ: Большие массивы лог-данных (big data) служат датасетами для обучения нейросетевых алгоритмов управления, навигации и компьютерного зрения.

4. Анализ производительности и диагностика: Пост полётный анализ логов позволяет оценить расход энергии, точность выдерживания маршрута, работу систем в различных режимах.

Структура лог-файла типичного БПЛА включает следующие потоки данных с метками времени (timestamp):

— **Сырые данные датчиков:** Показания гироскопов, акселерометров, магнитометра, барометра, сырые данные GPS (координаты, скорость, число спутников).

— **Оцененные состояния:** Ориентация (крен, тангаж, курс), положение (широта, долгота, высота), скорость (в глобальной и связанной системах координат), рассчитанные бортовым компьютером.

— **Задающие воздействия и управление:** Сигналы, отправляемые на двигатели и сервоприводы рулевых поверхностей (значения ШИМ, обороты).

— **Команды пилота/автопилота:** Заданные значения рычагов управления, целевые точки маршрута, команды режимов полета.

— **Диагностическая информация:** Состояние связи, напряжение батареи, внутренние ошибки автопилота, сообщения от других бортовых систем.

Для эффективной работы с цифровым двойником необходимы инструменты как для записи логов в ходе симуляции (полностью синтетические данные), так и для импорта и обработки логов реальных полетов. Популярные протоколы логирования в отрасли – это ULog (разработанный проектом PX4) и собственные форматы таких автопилотов, как ArduPilot.

1.5 Обработка данных инерциального устройства

Сырые данные с IMU не могут быть использованы напрямую для навигации из-за наличия шумов и дрейфов, описанных в пункте 3. Их обработка – это многоэтапный процесс фильтрации и сенсорной фьюжн

(объединения данных), целью которого является получение точной и устойчивой оценки ориентации, скорости и положения БПЛА.

Основные этапы и методы обработки:

1. **Предварительная фильтрация и калибровка:** на этом этапе к сырым данным применяются поправки на смещение, масштабный фактор и не ортогональность, полученные в ходе лабораторной калибровки. Также может применяться низкочастотная фильтрация для подавления высокочастотных вибрационных помех (например, с помощью БИХ- или КИХ-фильтров).

2. **Сенсорная фьюжн и оценка ориентации (AHRS):** это ядро обработки. Для объединения данных гироскопа (точных на коротких интервалах, но дрейфующих), акселерометра (дает вектор гравитации, но чувствителен к динамическим ускорениям) и магнитометра (дает курс, но подвержен магнитным помехам) используются различные алгоритмы:

— **Дополнительный фильтр (Complementary Filter):** Простой и вычислительно эффективный метод, разделяющий фильтрующий низкие частоты акселерометра/магнитометра и высокие частоты гироскопа.

— **Фильтр Калмана (Kalman Filter, KF) и его разновидности (Extended KF, Error-state KF):** наиболее распространенный и мощный подход. Это оптимальный в смысле минимума среднеквадратичной ошибки рекурсивный фильтр, который оценивает состояние системы (ориентация, смещения гироскопов) на основе потока зашумленных измерений. ЕКФ позволяет работать с нелинейной динамикой системы ориентации.

3. **Интегрирование в навигационное решение (INS):** после получения стабильной оценки ориентации (матрицы вращения или кватерниона) данные акселерометра могут быть преобразованы из связанной системы координат в глобальную (навигационную) и за вычетом гравитации проинтегрированы для получения скорости, а затем — повторно проинтегрированы для получения положения. Однако из-за двойного интегрирования даже небольшие смещения акселерометра приводят к

кубическому росту ошибки позиционирования со временем. Поэтому INS практически всегда корректируется внешними источниками:

- GPS/GNSS: Периодически сбрасывает накапливающуюся ошибку INS.
- Барометрическая высота/Высота по лазерному дальномеру.
- Коррекция по визуальной одометрии (VIO).

В цифровом двойнике процесс обработки данных IMU может тестироваться двумя способами:

- 1) как часть виртуального автопилота, обрабатывающего синтетические данные от модели датчиков, или
- 2) в режиме Hardware-in-the-Loop, когда реальный полетный контроллер (со своим программным обеспечением для обработки IMU) получает на свои входы синтетические сигналы от модели датчиков. Это позволяет проводить всестороннее тестирование навигационного контура [20][21][22][23].

1.6 Описание программ аналогов

Для создания и работы с цифровыми двойниками БПЛА существует широкий спектр программного обеспечения, от профессиональных коммерческих решений до open-source фреймворков.

1. Коммерческие и профессиональные среды моделирования:

— MATLAB/Simulink + Aerospace Blockset/ UAV Toolbox: Промышленный стандарт для моделирования динамики, проектирования систем управления и генерации кода. Позволяет создавать детализированные нелинейные модели, легко интегрирует алгоритмы обработки данных (фильтр Калмана) и предоставляет мощные средства анализа. Поддерживает HIL-тестирование.

— ANSYS SCADE Suite + Airbus Helicopters Simulation Models: Используется для разработки сертифицированного (DO-178C) ПО для критичных систем, включая моделирование [24].

— ESI Group, SCANeR studio: Комплексные решения для моделирования транспортных средств, включая БПЛА, с акцентом на реалистичную визуальную среду и сенсорное моделирование (камеры, лидары) [25].

2. Специализированные симуляторы БПЛА (преимущественно Open-Source):

— Gazebo + ROS (Robot Operating System) + PX4/ArduPilot: Фактически стандарт в научных исследованиях и разработке робототехники. Gazebo – это физический симулятор с продвинутым движком (ODE, Bullet) и моделями окружающей среды. ROS обеспечивает коммуникацию между модулями. Полетные стеки PX4 или ArduPilot могут работать внутри симуляции (Software-in-the-Loop, SITL), обеспечивая полный контур управления. Идеален для разработки автономных функций и алгоритмов SLAM/VIO.

— jMAVSIM / AirSim: Легковесный симулятор от разработчиков PX4. AirSim от Microsoft – это плагин для игрового движка Unreal Engine, предоставляющий высоко реалистичную графику и физику, а также модели камер, лидаров и IMU. Широко используется для исследований в области компьютерного зрения и машинного обучения [26].

— X-Plane (с плагином для связи с автопилотом): Использует высококачественную аэродинамическую модель на основе метода blade element theory. Может служить реалистичной средой для HIL-тестирования готовых автопилотов, подключаясь к ним по протоколу UDP [27].

3. Инструменты для анализа и визуализации полетных данных:

— FlightPlot, PlotJuggler (для PX4 ULog): Инструменты для графического анализа лог-файлов [28].

— Mission Planner, QGroundControl: Стандартные наземные контрольные станции для ArduPilot и PX4 соответственно. Позволяют не только управлять аппаратом, но и отображать телеметрию в реальном времени, анализировать логи после полета, планировать миссии [29].

Выбор конкретного программного стека зависит от задач: для академических исследований и быстрого прототипирования доминируют Gazebo/ROS и AirSim; для промышленной разработки и сертификации систем чаще используются MATLAB/Simulink и профессиональные решения.

2 ГЛАВА. РАЗРАБОТКА АРХИТЕКТУРЫ И МЕТОДОВ ПОСТРОЕНИЯ ЦИФРОВОГО ДВОЙНИКА

2.1 Получение данных БПЛА

Для создания адекватного цифрового двойника, способного с высокой точностью имитировать поведение реального беспилотного летательного аппарата, необходимо обеспечить полноценный сбор, структурирование, предварительную обработку и долгосрочное хранение данных. В рамках данной работы используется комбинированный подход к получению данных, основанный на двух источниках: данные реальных натурных полётов (физический БПЛА) и данные виртуальных полётов (цифровой двойник в режиме симуляции). Сопоставление этих двух потоков является основой для валидации моделей и калибровки параметров.

2.1.1 Классификация и структура данных БПЛА

Прежде чем перейти к описанию источников и методов сбора, необходимо определить, какие именно данные представляют ценность для цифрового двойника. Все данные, регистрируемые в процессе полёта (реального или виртуального), можно разделить на пять основных категорий:

1. Инерциальные и навигационные данные (высокочастотные, 50–1000 Гц): трёх осевые акселерометры (кажущееся ускорение в связанной системе координат); трёх осевые гироскопы (угловые скорости); трех осевой магнитометр (напряжённость магнитного поля Земли для определения курса); барометрическая высота (приведённая к уровню моря или относительная).

Эти данные являются первичными и поступают с частотой до 1 кГц в современных полётных контроллерах. Они наиболее чувствительны к шумам, вибрациям и температурным дрейфам, поэтому требуют тщательной предобработки.

2. Данные глобальной навигации (среднечастотные, 1–20 Гц): координаты (широта, долгота, абсолютная высота в системе WGS84); вектор

скорости (северная, восточная, вертикальная компоненты); количество видимых спутников и качество сигнала (HDOP/VDOP); дифференциальные поправки (при наличии RTK/PPK).

GPS-данные являются основным источником абсолютной привязки, но страдают от низкой частоты обновления, дискретности и возможных пропаданий сигнала.

3. Оценённые состояния (среднечастотные, 50–100 Гц): ориентация в углах Эйлера или кватернионах (крен, тангаж, рыскание); линейная скорость и положение в навигационной системе координат; смещения (bias) гироскопов и акселерометров, оценённые фильтром Калмана; диагностические флаги (режим полёта, статус сенсоров, целостность навигационного решения).

Эти данные являются результатом сенсорной фьюжн (объединения данных) на борту и представляют собой «очищенную» версию, пригодную для прямого анализа без глубокого понимания работы фильтра.

4. Управляющие сигналы и состояние системы управления (50–200 Гц): ШИМ-сигналы (широтно-импульсная модуляция) на каждый мотор или сервопривод; обороты двигателей (RPM) при наличии обратной связи; команды от оператора (значения газа, тангажа, крена, рыскания) или от автопилота (целевые углы, целевая высота, целевая скорость); активация режимов (стабилизация, удержание высоты, удержание позиции, автономный полёт по маршруту).

Эта категория данных критически важна для воспроизведения управляющих воздействий в цифровом двойнике и для анализа поведения системы управления в нештатных ситуациях.

5. Диагностическая и сервисная информация (низкочастотная, 1–10 Гц): напряжение и ток батареи (расчёт потребляемой мощности и остатка энергии); температура процессора и датчиков; состояние радиоканала (RSSI, качество пакетов); внутренние ошибки и предупреждения автопилота; время работы системы (аптайм).

Эти данные полезны для оценки энергоэффективности, анализа причин аварий и для общего мониторинга состояния аппарата.

2.1.2 Источники данных: реальный полёт (физический БПЛА)

При наличии физического прототипа мультироторного БПЛА сбор данных осуществляется с помощью бортового полётного контроллера. В рамках данной работы в качестве эталонной платформы используется контроллер Pixhawk 4 (или совместимый) под управлением прошивки PX4. Выбор обусловлен открытой архитектурой, широкой распространённостью в исследовательской среде и наличием детальной документации по форматам логов.

Организация логирования на борту состоит из: логирование осуществляется на microSD-карту (класс скорости не ниже UHS-I U3) в двоичном формате ULog, разработанном специально для PX4; частота записи настраивается через параметры: по умолчанию IMU пишется на 200 Гц (после антиалиасинговой фильтрации), GPS — на 10 Гц, управляющие сигналы — на 50 Гц; объём одного полёта продолжительностью 20 минут составляет примерно 150–250 МБ в зависимости от количества задействованных датчиков и частоты логирования; при включении режима «журналирование отладки» (debug logging) объём может возрасти до 500 МБ, что приемлемо для карт объёмом 32–64 ГБ.

Процедура сбора данных в натурном эксперименте:

1. Предполётная калибровка: перед вылетом выполняется статическая калибровка акселерометров, гироскопов и магнитометра (по стандартной процедуре QGroundControl). Результаты калибровки сохраняются в параметрах контроллера и автоматически учитываются при записи;

2. Запуск логирования: логирование начинается автоматически при включении питания контроллера (если настроено) или вручную через наземную станцию. Перед взлётом проверяется наличие microSD-карты и достаточность свободного места;

3. Выполнение полёта: полёт выполняется по заранее спланированному маршруту или в ручном режиме. Для валидации модели критически важно записывать несколько типов манёвров: зависание (hover), движение по прямой, повороты, набор и снижение высоты, имитацию отказов (например, выключение одного мотора на безопасной высоте);

4. Послеполётная выгрузка: после посадки и выключения контроллера microSD-карта извлекается, либо логи выгружаются через USB/телеметрию с помощью QGroundControl (функция «Download Flight Logs»). Логи сохраняются в локальную файловую систему компьютера исследователя.

Особенности работы с логами реальных полётов в том, что в логах могут присутствовать пропуски данных из-за кратковременной потери питания, ошибок записи на SD-карту или переполнения буфера. Также данные разных сенсоров имеют несинхронизированные временные метки (каждый сенсор имеет свой внутренний таймер). Для корректного анализа требуется приведение к единой временной шкале. Вибрации от моторов и пропеллеров наводят высокочастотный шум на акселерометры. Частота вибраций соответствует скорости вращения пропеллеров (обычно 100–300 Гц для квадрокоптеров среднего размера).

2.1.3 Источники данных: виртуальный полёт (цифровой двойник)

В режиме симуляции (SITL или HIL) цифровой двойник генерирует данные, полностью аналогичные по структуре данным реального полёта. Это достигается за счёт использования тех же протоколов и форматов (ULog), а также единой настройки частот логирования.

Генерация данных в симуляции состоит из математической модели динамики которая вычисляет истинное состояние аппарата: $\mathbf{x}_{true}(t)=[\mathbf{r},\mathbf{v},\mathbf{q},\boldsymbol{\omega}]$. Также модели сенсоров преобразуют истинное состояние в «сырые» показания, добавляя смещение нуля (bias), моделируемое случайным блужданием или постоянной величиной, белый шум с заданной спектральной

плотностью мощности, эффекты квантования АЦП (дискретизация по уровню) и температурный дрейф (при необходимости).

Затем эти сырые показания проходят через виртуальную версию бортового фильтра (ЕКФ из PX4), который выдаёт оценённые состояния. Все потоки записываются в тот же формат ULog, что и на реальном аппарате.

Преимуществами виртуального логирования это доступ к «истинным» значениям. В симуляции мы знаем точное положение, скорость и ориентацию в каждый момент времени. Это позволяет оценить ошибку оценки (разницу между истинным и оценённым состоянием) — величину, недоступную в реальном полёте. Также преимуществом виртуального логирования это повторяемость. Один и тот же сценарий может быть воспроизведён неограниченное количество раз с идентичными начальными условиями. Возможность безопасно моделировать катастрофические отказы (потеря мотора, разрушение пропеллера, полный отказ IMU) без риска уничтожения аппарата. Лёгкое изменение параметров модели, шумов сенсоров, погодных условий.

Ограничением виртуального логирования является то, что адекватность виртуальных данных полностью определяется точностью моделей динамики и сенсоров. Неучтённые физические эффекты (например, флаттер лопастей, интерференция воздушных потоков между винтами) могут привести к расхождению с реальностью.

2.2 Математическая модель динамики БПЛА

Для построения цифрового двойника разработана нелинейная математическая модель движения мультироторного БПЛА с шестью степенями свободы (6-DOF). Модель реализована в среде MATLAB/Simulink с возможностью экспорта в C++ для использования в ROS/Gazebo.

2.2.1 Системы координат и переменные состояния

Введены следующие системы координат:

- **Земная инерциальная система** $O_E X_E Y_E Z_E$ (NED: North-East-Down)

с началом в точке взлёта.

- **Связанная система** $O_B X_B Y_B Z_B$ с началом в центре масс БПЛА: X_B направлена вперёд, Y_B — вправо, Z_B — вниз.

Вектор состояния модели включает 12 переменных:

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, u, v, w, p, q, r]^T$$

где:

- x, y, z — координаты центра масс в инерциальной системе,
- ϕ, θ, ψ — углы крена, тангажа и рыскания (углы Эйлера),
- u, v, w — линейные скорости в связанной системе координат,
- p, q, r — угловые скорости вокруг осей X_B, Y_B, Z_B соответственно.

2.2.2 Уравнения динамики поступательного движения

Поступательное движение описывается вторым законом Ньютона в связанной системе координат с учётом силы тяжести и аэродинамических сил:

$$\begin{cases} \dot{u} = rv - qw - g \sin \theta + \frac{F_x}{m} \\ \dot{v} = pw - ru - g \cos \theta \sin \phi + \frac{F_y}{m} \\ \dot{w} = qu - pv - g \cos \theta \cos \phi + \frac{F_z}{m} \end{cases}$$

где m — масса БПЛА, g — ускорение свободного падения, F_x, F_y, F_z — проекции суммарной аэродинамической силы на оси связанной системы.

Для мультироторного аппарата основной вклад в F_z даёт суммарная тяга всех моторов, а в F_x, F_y — проекции тяги при наклоне аппарата и действие сил сопротивления.

2.2.3 Уравнения динамики вращательного движения

Изменение угловых скоростей связано с моментами, создаваемыми моторами и аэродинамическими силами:

$$\begin{cases} \dot{p} = \frac{1}{I_{xx}} (\tau_x - (I_{zz} - I_{yy})qr) \\ \dot{q} = \frac{1}{I_{yy}} (\tau_y - (I_{xx} - I_{zz})pr) \\ \dot{r} = \frac{1}{I_{zz}} (\tau_z - (I_{yy} - I_{xx})pq) \end{cases}$$

где I_{xx}, I_{yy}, I_{zz} — моменты инерции аппарата относительно главных осей, τ_x, τ_y, τ_z — суммарные моменты от моторов (относительно соответствующих осей).

2.2.4 Кинематические соотношения Эйлера

Связь между угловыми скоростями в связанной системе (p, q, r) и производными углов Эйлера:

$$\begin{cases} \dot{\phi} = p + q \sin \phi \tan \theta + r \cos \phi \tan \theta \\ \dot{\theta} = q \cos \phi - r \sin \phi \\ \dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta} \end{cases}$$

2.2.5 Модель силовой установки и управление

Каждый из n моторов (для квадрокоптера $n=4$) создаёт тягу T_i и реактивный момент Q_i :

$$T_i = k_T \cdot \omega_i^2, \quad Q_i = k_Q \cdot \omega_i^2$$

где ω_i — угловая скорость вращения i -го пропеллера, k_T и k_Q — коэффициенты тяги и момента, определяемые экспериментально.

Суммарные силы и моменты выражаются через матрицу смешивания **M**:

$$\begin{bmatrix} F_z \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = M \cdot \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix}$$

Для стандартного крестового квадрокоптера:

$$M = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -d & 0 & d \\ -d & 0 & d & 0 \\ -c & c & -c & c \end{bmatrix}$$

где d — плечо (расстояние от центра масс до оси мотора), $c=k_Q/k_T$ — коэффициент момента.

2.2.6 Моделирование аэродинамических помех

Для повышения адекватности в модель добавлены:

- **Соппротивление воздуха** — сила, пропорциональная квадрату скорости: $F_{drag} = -k_d \cdot v \cdot |v|$.
- **Ветровое возмущение** — добавка к скорости набегающего потока, моделируемая формирующим фильтром первого порядка (модель Драйдена).
- **Эффект земли** (при высоте менее 1 м) — увеличение эффективности пропеллеров, моделируемое эмпирической зависимостью.

Модель реализована в виде Simulink-блока UAV_6DOF_Dynamics с возможностью настройки параметров через структуру UAVParams.

2.3 Архитектура программного комплекса

Программный комплекс цифрового двойника реализован в виде веб-приложения с использованием платформы .NET и фреймворка [ASP.NET Core](#) [30]. Выбор этого технологического стека обусловлен его высокой производительностью, подтверждённой независимыми тестами, кроссплатформенностью, позволяющей развёртывание как на Windows, так и на Linux-серверах, а также богатой экосистемой для организации реального

времени через SignalR и возможностью использования единого языка C# как на серверной, так и на клиентской стороне при использовании Blazor [31][32][33]. Архитектура спроектирована с учётом требований многопользовательского режима, масштабируемости и безопасности, что особенно важно для промышленной эксплуатации системы в исследовательских и образовательных учреждениях.

В основе архитектуры лежат принципы чистой архитектуры и предметно-ориентированного проектирования. В соответствии с этими принципами всё решение разделено на несколько логических слоёв, каждый из которых имеет чётко определённые обязанности и не зависит от реализации нижележащих слоёв. Доменный слой содержит сущности предметной области, такие как симуляция, лог-файл, параметры полёта и метрики сравнения, а также интерфейсы репозитория и доменные события. Этот слой полностью свободен от каких-либо ссылок на внешние фреймворки или библиотеки, что делает его независимым от технологий хранения данных или доставки. Прикладной слой реализует варианты использования системы — команды и запросы, оформленные в стиле CQRS с использованием библиотеки MediatR. Именно здесь располагается вся бизнес-логика: запуск новой симуляции, загрузка и обработка логов реальных полётов, расчёт метрик сравнения, генерация отчётов. Инфраструктурный слой предоставляет конкретные реализации абстракций, объявленных в доменном и прикладном слоях. Сюда входят реализация репозитория на Entity Framework Core, сервис для работы с файловым хранилищем (локальным или S3-совместимым), парсеры логов в форматах ULog и CSV, а также сама математическая модель динамики БПЛА, написанная на C# с использованием библиотеки System.Numerics для векторных и кватернионных вычислений. Наконец, презентационный слой представляет собой [ASP.NET](#) Core WebAPI с контроллерами REST API, SignalR-хабом для реального времени и middleware для обработки аутентификации, логирования и обработки ошибок. Клиентское приложение может быть реализовано либо на Blazor WebAssembly (что позволяет запускать

C# код непосредственно в браузере), либо в виде отдельного одностраничного приложения на React или Vue.js, взаимодействующего с сервером через API и SignalR [34].

Математическая модель динамики, являющаяся сердцем цифрового двойника, реализована на C# в виде иерархии классов, поддерживающих три уровня детализации. Кинематическая модель с четырьмя степенями свободы описывает только движение центра масс и используется для быстрого прототипирования маршрутов и симуляции роя беспилотных аппаратов, где вычислительная эффективность важнее физической точности. Линеаризованная модель с шестью степенями свободы использует постоянные коэффициенты в уравнениях Эйлера и применяется для первичной настройки ПИД-регуляторов стабилизации. Полная нелинейная модель, также с шестью степенями свободы, интегрирует уравнения движения методом Рунге-Кутты четвёртого порядка, учитывает эффект земли, нелинейное аэродинамическое сопротивление, ветровые возмущения в виде модели Драйдена и температурные дрейфы сенсоров. Эта модель используется для финальной валидации алгоритмов управления и моделирования отказных ситуаций. Модель сенсоров добавляет к истинным значениям, получаемым из динамической модели, реалистичные шумы (белый гауссовский шум со спектральной плотностью, соответствующей реальным датчикам), медленно меняющиеся смещения нуля, моделируемые случайным блужданием, и эффекты квантования аналого-цифрового преобразователя. Все компоненты модели написаны на чистом C# с акцентом на вычислительную эффективность, что позволяет достигать частоты обновления до двух тысяч герц на современном серверном оборудовании.

Для организации обмена данными в реальном времени между сервером и клиентами используется библиотека SignalR, которая автоматически выбирает оптимальный транспорт (WebSockets, Server-Sent Events или Long Polling) в зависимости от возможностей браузера и сетевых условий. SignalR-хаб предоставляет два основных метода: присоединение к определённой

симуляции по её идентификатору и отправка команд управления (пауза, возобновление, внедрение отказа). При запуске симуляции клиент получает от сервера уникальный URL для WebSocket-соединения и устанавливает постоянный канал, по которому сервер передаёт телеметрические пакеты с частотой до тридцати герц. Каждый пакет содержит временную метку, текущее положение аппарата в трёхмерном пространстве, его ориентацию в углах Эйлера или кватернионах, линейную и угловую скорости, показания виртуальных сенсоров, а также величину рассогласования между виртуальной траекторией и реальным логом (если симуляция запущена в режиме сравнения). SignalR также обеспечивает автоматическое переподключение при временных разрывах связи и сохраняет состояние последней переданной телеметрии, что позволяет клиенту бесшовно продолжить отображение после восстановления соединения.

Управление жизненным циклом симуляций реализовано через фоновый сервис, работающий на протяжении всего времени работы веб-приложения. Этот сервис использует потокобезопасную коллекцию для хранения активных симуляций и неблокирующую очередь для постановки новых задач. При поступлении команды на запуск симуляции в очередь помещается рабочий элемент, содержащий идентификатор симуляции, тип модели, параметры окружающей среды и опциональный идентификатор реального лога для сравнения. Фоновый сервис извлекает элементы из очереди, создаёт для каждого новый экземпляр модели динамики через фабрику, загружает из базы данных необходимые конфигурационные параметры и запускает выполнение в отдельной задаче. Каждая симуляция выполняется в собственном контексте, изолированном от остальных, что предотвращает влияние одной симуляции на производительность другой. По окончании симуляции фоновый сервис вызывает модуль сравнения для расчёта метрик, обновляет статус симуляции в базе данных и отправляет клиенту финальное сообщение с результатами. Если симуляция была прервана из-за ошибки или превышения максимальной

длительности, сервис также корректно обрабатывает эту ситуацию, освобождая ресурсы и уведомляя пользователя.

Модуль сравнения представляет собой отдельный сервис, который принимает на вход два временных ряда реальную траекторию из загруженного лога и смоделированную траекторию из виртуального полёта. Поскольку временные метки в этих рядах могут не совпадать, первым шагом выполняется синхронизация: реальная траектория интерполируется на временную сетку виртуальной с помощью кусочно-линейной интерполяции или сплайнов третьего порядка. После синхронизации вычисляется набор метрик, включающих среднеквадратичную ошибку позиции в трёхмерном пространстве, максимальную абсолютную ошибку ориентации по каждому из углов Эйлера, интегральную абсолютную ошибку скорости, расстояние Хаусдорфа между множествами точек траектории как меру максимального расхождения, а также временную задержку между сигналами, вычисляемую через кросс-корреляцию. Для вычисления этих метрик сервис использует библиотеку [Math.NET](#) Numerics, предоставляющую эффективные реализации линейной алгебры, интегрирования и статистических функций. Результаты сравнения сохраняются в базе данных в формате JSON вместе с идентификатором симуляции, что позволяет в любой момент запросить историю и построить графики изменения точности модели при варьировании параметров.

Хранение данных организовано комбинированным способом. Реляционная база данных PostgreSQL (или Microsoft SQL Server) через Entity Framework Core хранит метаданные о пользователях, симуляциях, загруженных логах и результатах сравнения. Для повышения производительности запросов созданы индексы по полям UserId, Status и CreatedAt, а также настроено полнотекстовое индексирование для поиска по имени файла или описанию симуляции. Сами временные ряды как загруженные реальные логи, так и сгенерированные виртуальные траектории сохраняются в колоночном формате Parquet со сжатием Snappy, который

обеспечивает высокую степень компрессии (обычно в пять-десять раз по сравнению с CSV) и позволяет выполнять фильтрацию по временному диапазону без загрузки всего файла в память. Parquet-файлы хранятся либо в локальной файловой системе сервера, либо в облачном объектном хранилище (совместимом с S3), что обеспечивает возможность масштабирования и резервного копирования. Исходные загруженные пользователем логи в форматах ULog, BIN или CSV также сохраняются для возможности повторной обработки в случае изменения алгоритмов предобработки или калибровки.

Клиентская часть веб-приложения реализована на Blazor WebAssembly, что позволяет выполнять код на C# непосредственно в браузере пользователя. Такой подход даёт возможность переиспользовать между сервером и клиентом модели данных, валидационные атрибуты и даже некоторые алгоритмы предобработки. Пользовательский интерфейс построен с использованием компонентной библиотеки MudBlazor, реализующей принципы Material Design и обеспечивающей адаптивный дизайн для экранов различных размеров. Трёхмерная визуализация полёта выполняется с помощью библиотеки Three.js, которая инициализируется и управляется через механизм JSInterop — мост между управляемым кодом C# и JavaScript-окружением браузера. При загрузке страницы инициализируется сцена с камерой, освещением, сеткой координат и загружается трёхмерная модель беспилотного аппарата в формате GLTF или GLB. В процессе симуляции каждое полученное через SignalR телеметрическое сообщение приводит к обновлению положения и ориентации модели в пространстве, добавлению новой точки к линиям траекторий (реальная траектория отображается красным цветом, виртуальная — синим или зелёным), а также к перерисовке дашборда с числовыми значениями параметров полёта. Пользователь имеет возможность переключать режим камеры (следование за аппаратом, свободное вращение, вид сверху), приближать и отдалять сцену, а также сохранять текущий вид в графический файл для последующего включения в отчёт.

Безопасность веб-приложения обеспечивается на нескольких уровнях. Аутентификация пользователей реализована через [ASP.NET](#) Core Identity с хранением учётных записей в PostgreSQL и поддержкой двухфакторной аутентификации для пользователей с повышенными привилегиями. Для доступа к API и SignalR-хабу используется JWT (JSON Web Token), который выдаётся при успешном входе в систему и должен передаваться во всех последующих запросах в заголовке Authorization. SignalR поддерживает передачу токена через строку запроса при установке WebSocket-соединения, что особенно удобно для клиентских приложений. На уровне авторизации реализована ролевая модель: обычные пользователи могут запускать симуляции, загружать логи и просматривать только свои собственные результаты; исследователи имеют доступ к расширенным метрикам и могут сохранять симуляции как шаблоны; администраторы управляют пользователями, просматривают все симуляции и настраивают параметры системы (например, максимальное количество параллельных симуляций на сервере). Все загружаемые файлы логов проходят валидацию по расширению, магическому числу (сигнатуре) и размеру, а также сканируются на вредоносное содержимое, что предотвращает атаки через загрузку исполняемых файлов или архивов-бомб.

Развёртывание системы поддерживает два основных режима. Для разработки и небольших инсталляций используется Docker Compose, который поднимает контейнеры с PostgreSQL, Redis (для бэкаплейна SignalR при горизонтальном масштабировании), серверным приложением [ASP.NET](#) Core и статическими файлами Blazor WebAssembly, обслуживаемыми через Nginx. Этот режим удобен для быстрого старта и тестирования на локальной машине. Для промышленной эксплуатации в облаке или корпоративном дата-центре подготовлены манифесты для Kubernetes с автоматическим масштабированием подов бэкенда в зависимости от загрузки CPU и количества активных SignalR-соединений. В этом режиме база данных PostgreSQL выносятся на управляемый экземпляр, а файловое хранилище

Parquet заменяется на S3-совместимое объектное хранилище, что обеспечивает отказоустойчивость и возможность горизонтального масштабирования до десятков параллельных симуляций. В обоих режимах предусмотрены liveness-пробы для автоматического перезапуска зависших компонентов и readiness-пробы, гарантирующие, что новый под не начнёт принимать трафик до полной инициализации всех зависимостей.

Мониторинг работоспособности и производительности системы осуществляется с использованием стека Prometheus и Grafana. ASP.NET Core приложение экспортирует метрики в формате, понятном Prometheus, включая количество HTTP-запросов по эндпоинтам, время их выполнения, количество активных соединений SignalR, загрузку процессора и памяти, а также кастомные метрики — число запущенных симуляций, количество ошибок модели, среднюю частоту обновления телеметрии. На основе этих метрик в Grafana построены дашборды для администратора, отображающие текущую загрузку системы, долю успешных симуляций и предупреждения о превышении пороговых значений. Логирование событий ведётся через Serilog с записью в структурированном формате JSON и отправкой в централизованную систему агрегации логов (например, Elasticsearch или Loki), что позволяет выполнять поиск по всем компонентам системы и анализировать причины сбоев.

Таким образом, разработанная архитектура веб-приложения на C# ASP.NET Core представляет собой законченное, масштабируемое и безопасное решение для создания цифровых двойников беспилотных летательных аппаратов. Она охватывает все этапы работы с системой: от аутентификации пользователя и загрузки реальных полётных данных до запуска высокоточных симуляций, визуализации их результатов в реальном времени и генерации аналитических отчётов. Использование современного стека технологий, включающего .NET 8, SignalR, Blazor WebAssembly и Three.js, обеспечивает как высокую производительность вычислений на сервере, так и богатый интерактивный опыт на стороне клиента. Модульность

и чёткое разделение слоёв позволяют легко модифицировать отдельные компоненты (например, добавить новую модель динамики или другой формат импорта логов) без нарушения работы остальной части системы.

Список использованных источников

1. БПЛА: что это такое, виды беспилотников и законы их применения в России [Электронный ресурс]: официальный сайт РБК.
<https://www.rbc.ru/base/17/02/2026/6993763e9a7947359dde6462> (дата обращения 06.05.2026).
2. Цифровые двойники: превращаем данные в деньги с помощью матмоделирования [Электронный ресурс]: официальный сайт Хабр.
<https://habr.com/ru/companies/ibs/articles/897072/> (дата обращения 06.05.2026).
3. Тяжела и неказиста жизнь простого RnD. Часть первая: как работают с новыми технологиями в крупных компаниях [Электронный ресурс]: официальный сайт Хабр.
https://habr.com/ru/companies/ru_mts/articles/863054/ (дата обращения 06.05.2026).
4. Машинное обучение (ML): возможности и сценарии применения [Электронный ресурс]: официальный сайт Yandex Cloud.
https://yandex.cloud/ru/blog/machine-learning-guide?&utm_referrer=https%3A//yandex.ru/ (дата обращения 06.05.2026).
5. TechnologiCS – цифровизация жизненного цикла продукции на базе одной системы [Электронный ресурс]: официальный сайт Хабр.
<https://habr.com/ru/companies/nanosoft/articles/530058/> (дата обращения 06.05.2026).
6. Математическая модель движения мультироторного беспилотного летательного аппарата – тема научной статьи по механике и машиностроению читайте бесплатно текст научно-исследовательской работы в электронной библиотеке [Электронный ресурс]: официальный сайт КиберЛенинка. <https://cyberleninka.ru/article/n/matematicheskaya-model-dvizheniya-multirotornogo-bespilotnogo-letatel'nogo-apparata> (дата обращения 06.05.2026).

7. Hardware-in-the-Loop [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/companies/itelma/articles/489422/> (дата обращения 06.05.2026).

8. Что такое «модель в цикле» (MITL) и почему это важно? [Электронный ресурс]: официальный сайт Macgence AI. <https://ru.macgence.com/blog/what-is-model-in-the-loop-mitl-and-why-does-it-matter/> (дата обращения 06.05.2026).

9. Software-in-the-Loop Explained | Lower Development Costs and Strengthen Code for Faster Launch [Электронный ресурс]: официальный сайт OPAL-RT. <https://www.opal-rt.com/blog/what-is-software-in-the-loop/> (дата обращения 06.05.2026).

10. Inertial Measurement Unit (IMU) [Электронный ресурс]: официальный сайт An Introduction. <https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/> (дата обращения 06.05.2026).

11. GPS: что это такое, как работает, чем отличается от ГЛОНАСС [Электронный ресурс]: официальный сайт РБК Тренды. <https://trends.rbc.ru/trends/innovation/672208cf9a7947165684ff0a> (дата обращения 06.05.2026).

12. Простая имитационная модель мобильного робота в ROS и Gazebo [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/articles/820035/> (дата обращения 06.05.2026).

13. Быстрая разработка для микроконтроллеров в Simulink на примере полифункционального зарядника [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/articles/692592/> (дата обращения 06.05.2026).

14. Моделирование динамики консольных фрагментов упругих лопастей Н-роторов Дарье в потоке воздуха – тема научной статьи по механике и машиностроению читайте бесплатно текст научно-исследовательской работы в электронной библиотеке [Электронный ресурс]: официальный сайт КиберЛенинка.

<https://cyberleninka.ru/article/n/modelirovanie-dinamiki-konsolnyh-fragmentov-uprugih-lopastey-n-rotorov-darie-v-potoke-vozduha?ysclid=mpb0v9o8og334085020> (дата обращения 06.05.2026).

15. What is Plant Modeling? [Электронный ресурс]: официальный сайт Precog. <https://precog.co/glossary/plant-modeling/> (дата обращения 06.05.2026).

16. Математическая модель сенсора с открытой областью пространства – тема научной статьи по электротехнике, электронной технике, информационным технологиям читайте бесплатно текст научно-исследовательской работы в электронной библиотеке [Электронный ресурс]: официальный сайт КиберЛенинка. <https://cyberleninka.ru/article/n/matematicheskaya-model-sensora-s-otkrytoy-oblastyu-prostranstva?ysclid=mpb0yikmiz835614592>

17. Development of Environment Model Specifications — Klever 4.0.1 documentation [Электронный ресурс]: официальный сайт Klever. https://klever.readthedocs.io/en/latest/dev_env_model_specs.html (дата обращения 06.05.2026).

18. моделирование системы управления автопилота самолета в средах SciLab и Simintech – тема научной статьи по компьютерным и информационным наукам читайте бесплатно текст научно-исследовательской работы в электронной библиотеке [Электронный ресурс]: официальный сайт КиберЛенинка. <https://cyberleninka.ru/article/n/modelirovanie-sistemy-upravleniya-avtopilota-samoleta-v-sredah-scilab-i-simintech?ysclid=mpb11syujh950083441>

19. Вычислительная гидродинамика. [Электронный ресурс]: официальный сайт Рувики. https://ru.ruwiki.ru/wiki/Вычислительная_гидродинамика (дата обращения 06.05.2026).

20. Система определения положения и курса (AHRS) [Электронный ресурс]: официальный сайт SENSSET. <https://sensset.ru/docs/integrated-motion-navigation-sensors-attitude-heading-reference-system-ahrs/> (дата обращения 06.05.2026).

21. Комплементарный фильтр [Электронный ресурс]: официальный сайт Класс робототехники. <https://robotclass.ru/articles/complementary-filter/> (дата обращения 06.05.2026).

22. Фильтр Калмана [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/articles/166693/> (дата обращения 06.05.2026).

23. Система навигации транспортного средства на основе интегрированной ИНС/СНС/Одометрии – тема научной статьи по электротехнике, электронной технике, информационным технологиям читайте бесплатно текст научно-исследовательской работы в электронной библиотеке [Электронный ресурс]: официальный сайт КиберЛенинка. <https://cyberleninka.ru/article/n/sistema-navigatsii-transportnogo-sredstva-na-osnove-integrirovannoy-ins-sns-odometrii> (дата обращения 06.05.2026).

24. Ansys SCADe [Электронный ресурс]: официальный сайт CADIT Global Official. <https://www.cadit.com/products/ansys-scade> (дата обращения 06.05.2026).

25. ESI Group: деятельность, чем известна [Электронный ресурс]: официальный сайт РУВИКИ. https://ru.ruwiki.ru/wiki/ESI_Group (дата обращения 06.05.2026).

26. AirSim Simulation [Электронный ресурс]: официальный сайт PX4 Guide. https://docs.px4.io/main/en/sim_airsim/ (дата обращения 06.05.2026).

27. The world's most advanced flight simulator. [Электронный ресурс]: официальный сайт X-Plane. <https://www.x-plane.com/> (дата обращения 06.05.2026).

28. Flight Log Analysis [Электронный ресурс]: официальный сайт PX4 Guide. https://docs.px4.io/main/en/log/flight_log_analysis (дата обращения 06.05.2026).

29. Для чего используются Mission Planner, QGC, Pulsar и DJI GO? [Электронный ресурс]: официальный сайт PCHARDWAREPRO.COM. <https://www.pchardwarepro.com/ru/Для-чего-используются-Mission-Planner--QGroundControl--Pulsar-и-DJI-Go-Assistant-2-при-управлении-дроном/> (дата обращения 06.05.2026).

30. .NET 10 и C# 14: что поменяется в вашем коде [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/articles/968538/> (дата обращения 06.05.2026).

31. Windows Server или Linux-дистрибутивы? Выбираем серверную ОС [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/companies/ruvds/articles/499322/> (дата обращения 06.05.2026).

32. SignalR в помощь, или как оживить web [Электронный ресурс]: официальный сайт Хабр. https://habr.com/ru/companies/dnevnik_ru/articles/167307/ (дата обращения 06.05.2026).

33. Blazor | Создание клиентских веб-приложений с помощью C# [Электронный ресурс]: официальный сайт .NET. <https://dotnet.microsoft.com/ru-ru/apps/aspnet/web-apps/blazor> (дата обращения 06.05.2026).

34. React vs Vue – подробное сравнение и перспективы [Электронный ресурс]: официальный сайт Хабр. <https://habr.com/ru/articles/904698/> (дата обращения 06.05.2026).